

API DESIGN

IPS18 — Automated Healthcare Document Processing and Intelligence Platform

Track A — Artificial Intelligence & Machine Learning
B.E. Final Year Capstone Project — Semester 7

1. Introduction

This document specifies the REST API surface of the IPS18 platform. The API is organized around the eight modules defined in the companion Module Design document, so that every endpoint maps to a clear, single-responsibility service. All endpoints are versioned under a common prefix and return JSON, allowing the frontend, the RAG assistant, and any future third-party integration to interact with the platform through a consistent, predictable contract.

The design deliberately keeps AI-heavy operations — OCR, classification, extraction, summarization, and retrieval — behind simple, resource-oriented endpoints. This means the complexity of the underlying models is fully hidden from API consumers: a client only ever asks for a document's entities, summary, or search results, without needing to know which model produced them or how.

1.1 Base URL and Versioning

All endpoints are served under the prefix shown below. Versioning is applied at the URL level so that future breaking changes can be introduced as a new version without disrupting existing integrations.

Base URL: `/api/v1`

2. Common Conventions

The following conventions apply uniformly across every endpoint described in this document, and are established here once rather than repeated per endpoint.

Aspect	Convention
Authentication	Bearer token (JWT) required on all endpoints except the health check.
Pagination	List endpoints accept <code>?page=</code> and <code>?page_size=</code> ; responses include a <code>total_count</code> field.
Error format	<code>{ "error": { "code": "OCR_FAILED", "message": "..." } }</code> , paired with a matching HTTP status code.
Status codes	200 success, 201 created, 202 accepted (async processing started), 400/401/403/404/422/500 standard error cases.

Aspect	Convention
Versioning	URL-based (/api/v1/...), allowing non-breaking evolution as the platform matures.

Table 2.1 — API-wide conventions

3. API Endpoints by Module

3.1 M1 — Ingestion

These endpoints handle the entry of a new document into the platform and let a client check on its progress without needing to poll the entire pipeline manually.

POST /api/v1/documents

Uploads a new healthcare document for processing. The request is multipart, since it carries a binary file alongside optional metadata such as the associated patient.

```
Request: multipart/form-data
  file: <PDF|JPG|JPEG|PNG>
  patient_id: (optional)

{
  "document_id": "doc_8f2a1c",
  "status": "Pending",
  "uploaded_at": "2026-08-16T10:00:00Z"
}
```

Errors: 400 (unsupported file format), 413 (file exceeds size limit).

GET /api/v1/documents/{document_id}/status

Returns the current pipeline stage and status for a given document, allowing the frontend to show live processing progress.

```
{
  "document_id": "doc_8f2a1c",
  "status": "Processing",
  "stage": "classification"
}
```

3.2 M2 — OCR & Preprocessing

This endpoint exposes the OCR output for a document once preprocessing has completed. The OCR confidence score included in the response is deliberately surfaced here, since it is used both by the frontend (to flag low-quality scans) and by downstream evaluation of the extraction module.

GET /api/v1/documents/{document_id}/text

Retrieves the OCR'd raw text for a document, along with layout metadata and a confidence score.

```
{
  "document_id": "doc_8f2a1c",
  "raw_text": "Patient: ... Rx: Metformin 500mg ...",
  "ocr_confidence": 0.87,
  "layout": { "pages": 1, "tables_detected": 1 }
}
```

Note: the noise-injection pipeline used to construct the noisy-OCR training dataset (described in the project's Zeroth Review report) is an internal, offline training-time tool. It is not exposed as a runtime API, since it has no role in serving live document requests.

3.3 M3 — Classification

A single read endpoint exposes the predicted document category once classification has run, which the frontend uses to route the document to the correct extraction and display logic.

GET `/api/v1/documents/{document_id}/classification`

Returns the predicted document type and the model's confidence in that prediction.

```
{
  "document_id": "doc_8f2a1c",
  "document_type": "prescription",
  "confidence": 0.94
}
```

3.4 M4 — Medical Information Extraction

This is the API surface over the platform's core research module. Every extracted entity carries a confidence score and a character-level source offset — the offset is what allows the Source Reference and Verification module to later highlight exactly where in the original document a value came from.

GET `/api/v1/documents/{document_id}/entities`

Returns all clinical entities extracted from a document, with type, value, confidence, and source location.

```
{
  "document_id": "doc_8f2a1c",
  "entities": [
    {
      "type": "MEDICATION",
      "value": "Metformin",
      "confidence": 0.91,
      "source_offset": { "start": 142, "end": 151 }
    },
    {
      "type": "DOSAGE",
      "value": "500mg",
      "confidence": 0.88,
      "source_offset": { "start": 152, "end": 157 }
    }
  ]
}
```

POST `/api/v1/documents/{document_id}/entities/{entity_id}/review`

Allows an authorized reviewer to correct a low-confidence extraction before it is finalized. This supports the human-in-the-loop review capability planned as a future enhancement.

Request Body:

```
{ "corrected_value": "Metformin 500mg", "reviewer_id": "user_12" }

{ "status": "updated" }
```

3.5 M5 — Summarization

A single endpoint returns the generated summary for a document, together with a faithfulness score indicating how well the summary is supported by the source content.

GET `/api/v1/documents/{document_id}/summary`

Retrieves the AI-generated summary for a document.

```
{
  "document_id": "doc_8f2a1c",
  "summary": "Patient prescribed Metformin 500mg for type 2 diabetes management...",
  "faithfulness_score": 0.93
}
```

3.6 M6 — Search & RAG Q&A

This endpoint is the platform's conversational interface. It accepts a natural-language question, optionally scoped to a patient, and returns an answer that is grounded in retrieved document content — every answer carries the specific source references that support it, so it can be verified rather than trusted blindly.

POST `/api/v1/search`

Accepts a natural-language query and returns a retrieval-grounded answer with source references.

```
Request Body:
{ "query": "What medication was the patient on last visit?", "patient_id": "pat_004" }

{
  "answer": "The patient was prescribed Metformin 500mg twice daily.",
  "sources": [
    { "document_id": "doc_8f2a1c", "chunk_id": "c_3", "offset": { "start": 142, "end": 157 } }
  ],
  "confidence": 0.89
}
```

3.7 M7 — Document & Processing Management

These endpoints support the day-to-day operational view of the platform — browsing the document repository, filtering by patient or status, and removing documents when necessary.

GET `/api/v1/documents`

Lists and searches the document repository. Supports filtering and pagination.

Query parameters: `patient_id`, `document_type`, `status`, `page`, `page_size`

```
{
  "documents": [ { "document_id": "doc_8f2a1c", "status": "Completed" } ],
  "total_count": 1
}
```

DELETE `/api/v1/documents/{document_id}`

Removes a document from the active repository. Implemented as a soft delete in line with data retention requirements.

```
{ "status": "deleted" }
```

3.8 M8 — Source Reference & Verification

This endpoint is called by the frontend every time it displays an AI-generated value — whether an extracted entity, a summary sentence, or a search answer. Given the offset that produced that value, it resolves the exact page and highlighted region in the original document, which is what makes the platform's verification-by-source-reference behavior possible.

GET `/api/v1/documents/{document_id}/reference?offset_start=142&offset_end=157`

Resolves a text offset back to its exact location in the original document for on-screen highlighting.

```
{
  "document_id": "doc_8f2a1c",
  "page": 1,
  "highlighted_region": { "x": 120, "y": 340, "width": 200, "height": 18 },
  "snippet": "Metformin 500mg"
}
```

4. Traceability to Functional Requirements

Every functional requirement defined in the PRD is implemented by exactly one module and exposed through one or more of the endpoints above. This traceability is what allows the platform to be verified as fully specified end-to-end, and is the kind of evidence a Gate 6 review checks for when confirming a system is operable from documentation alone.

FR Range	Module	Endpoint(s)
FR-001 – FR-004	M1	POST /documents, GET /documents/{id}/status
FR-005 – FR-008	M2	GET /documents/{id}/text
FR-009 – FR-011	M3	GET /documents/{id}/classification
FR-012 – FR-015	M4	GET /documents/{id}/entities
FR-016 – FR-018	M5	GET /documents/{id}/summary
FR-019 – FR-022	M6	POST /search
FR-023 – FR-026	M7	GET /documents, DELETE /documents/{id}
FR-027 – FR-029	M8	GET /documents/{id}/reference

Table 4.1 — Functional requirement to API traceability